

## Lab Assignment 13.3

### AI Assisted Coding

2303A51399

[BT.NO:](#) 06

---

## Task 1: Refactoring – Removing Code Duplication

**Objective:** To eliminate repeated logic by extracting reusable functions.

**Question:** Refactor the legacy script that calculates the area and perimeter of rectangles using repeated print statements.

### AI Prompt:

"Refactor the following legacy Python code to eliminate duplication. Create a reusable function with proper docstrings to calculate the area and perimeter of a rectangle. Use the dimensions (5, 10), (7, 12), and (10, 15) provided in the starter code ."

### Refactored Code

Python

```
def calculate_rectangle_metrics(length, width):  
    """  
    Calculates and prints the area and perimeter of a rectangle.  
  
    Args:  
        length (int/float): The length of the rectangle.  
        width (int/float): The width of the rectangle.  
    """  
    area = length * width  
    perimeter = 2 * (length + width)  
    print(f"Area of Rectangle: {area}")  
    print(f"Perimeter of Rectangle: {perimeter}")  
  
# List of rectangle dimensions (length, width)  
rectangle_data = [(5, 10), (7, 12), (10, 15)]  
  
# Iterate through data to remove repetitive print calls
```

```
for length, width in rectangle_data:  
    calculate_rectangle_metrics(length, width)
```

### Output:

Plaintext

```
Area of Rectangle: 50  
Perimeter of Rectangle: 30  
Area of Rectangle: 84  
Perimeter of Rectangle: 38  
Area of Rectangle: 150  
Perimeter of Rectangle: 50
```

**Code Explanation:** The refactored code replaces six independent print statements with a single modular function, `calculate_rectangle_metrics`. By storing the dimensions in a list of tuples and using a `for` loop, the script achieves the "Expected Outcome" of removing duplicated code blocks. A Google-style docstring was added to satisfy the requirement for proper documentation.

---

## Task 2: Refactoring – Optimizing Loops and Conditionals

**Objective:** To improve performance by replacing inefficient nested loops with optimized structures.

**Question:** Refactor a script that checks for the presence of elements using a nested loop structure.

### AI Prompt:

"The following code uses a nested loop to search for names, which results in  $O(n \times m)$  complexity. Refactor this using a Python set to optimize the lookup to  $O(1)$  average time complexity."

### Refactored Code

Python

```
import time  
  
names = ["Alice", "Bob", "Charlie", "David"]  
search_names = ["Charlie", "Eve", "Bob"]  
  
# Optimization: Convert the search list into a set for O(1) lookups
```

```
names_set = set(names)

start_time = time.perf_counter()
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.perf_counter()

print(f"Refactored execution time: {end_time - start_time:.6f} seconds")
```

### Output:

#### Plaintext

```
Charlie is in the list
Eve is not in the list
Bob is in the list
Refactored execution time: 0.000042 seconds
```

**Code Explanation:** The legacy code used a nested loop (a loop inside a loop) to check membership, which is inefficient for large datasets. The refactored version converts the `names` list into a `set`. In Python, checking membership in a set has an average time complexity of  $O(1)$ , significantly improving performance over the  $O(n)$  complexity of a list search.

---

## Task 3: Refactoring – Extracting Reusable Functions

**Objective:** To modularize code by extracting calculations into reusable functions.

**Question:** Refactor a script where price and tax calculations are written inline.

### AI Prompt:

"Refactor this legacy script by creating a function called `calculate_total(price)`. The function should handle the 18% tax calculation and return the total value. Include proper documentation."

### Refactored Code

#### Python

```
def calculate_total(price, tax_rate=0.18):
```

```
"""
```

Calculates the total price after applying a standard tax rate.

Args:

price (float): The base price of the item.

tax\_rate (float): The decimal tax rate (default 0.18).

Returns:

float: Total price including tax.

```
"""
```

```
tax = price * tax_rate
```

```
return price + tax
```

```
# Applying the reusable function to different price points
```

```
prices = [250, 500]
```

```
for p in prices:
```

```
    total_price = calculate_total(p)
```

```
    print("Total Price:", total_price)
```

## Output:

Plaintext

Total Price: 295.0

Total Price: 590.0

**Code Explanation:** This task transforms "inline" logic into a "modular" structure. Instead of recalculating the tax manually for every price variable, the `calculate_total` function centralizes the logic. This makes the code easier to update; for instance, if the tax rate changes, it only needs to be updated in one place.

---

## Task 4: Refactoring – Replacing Hardcoded Values with Constants

**Objective:** To improve maintainability by replacing "magic numbers" with named constants.

**Question:** Use AI to identify hardcoded values in a circle calculation script and replace them with constants.

**AI Prompt:**

"Identify the hardcoded values in this script (3.14159 and 7). Refactor the code to use named constants like PI and RADIUS to improve maintainability."

## Refactored Code

Python

```
# Named constants to replace magic numbers
PI = 3.14159
RADIUS = 7

def calculate_circle_properties(r):
    """Calculates area and circumference for a given radius."""
    area = PI * (r ** 2)
    circumference = 2 * PI * r
    return area, circumference

area, circ = calculate_circle_properties(RADIUS)

print(f"Area of Circle: {area}")
print(f"Circumference of Circle: {circ}")
```

### Output:

Plaintext

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

**Code Explanation:** The legacy code used "magic numbers" (3.14159) directly in the print statements. This is a "code smell" because it makes the script harder to maintain. The refactored code defines `PI` and `RADIUS` at the top of the script. If the precision of `PI` needs to be increased later, it only requires changing one line.

---

## Task 5: Refactoring – Improving Variable Naming and Readability

**Objective:** To enhance readability using descriptive variable names and comments.

**Question:** Refactor a script with unclear variable names (a, b, c) into a readable format .

### AI Prompt:

"Refactor this script: 'a = 10; b = 20; c = a \* b / 2' . Rename the variables to be descriptive of a triangle's area calculation and add inline comments explaining the logic."

## Refactored Code

Python

```
# The base of the triangle in units
base = 10

# The perpendicular height of the triangle
height = 20

# Calculate the area using the formula: (Base * Height) / 2
triangle_area = (base * height) / 2

# Output the calculated result
print(triangle_area)
```

### Output:

Plaintext

100.0

**Code Explanation:** The original script used single-letter variables (a, b, c) which provided no context . The refactored version uses `base`, `height`, and `triangle_area`. While the logic remains identical to ensure the output does not change , the code is now self-documenting and much easier for a human to read and understand.

---

Would you like me to generate a summary table comparing the performance of the nested loops versus the set lookups for Task 2?